

If It Looks Like a Rootkit and Deceives Like a Rootkit: A Critical Examination of Kernel-Level Anti-Cheat Systems

Christoph Dorner

christoph.dorner@fhstp.ac.at
St. Pölten University of Applied Sciences
St. Pölten, Austria

Lukas Daniel Klausner

mail@l17r.eu
St. Pölten University of Applied Sciences
St. Pölten, Austria

ABSTRACT

Addressing a critical aspect of cybersecurity in online gaming, this paper systematically evaluates the extent to which kernel-level anti-cheat systems mirror the properties of rootkits, highlighting the importance of distinguishing between protective and potentially invasive software. After establishing a definition for rootkits (making distinctions between rootkits and simple kernel-level applications) and defining metrics to evaluate such software, we introduce four widespread kernel-level anti-cheat solutions. We lay out the inner workings of these types of software, assess them according to our previously established definitions, and discuss ethical considerations and the possible privacy infringements introduced by such programs. Our analysis shows two of the four anti-cheat solutions exhibiting rootkit-like behaviour, threatening the privacy and the integrity of the system. This paper thus provides crucial insights for researchers and developers in the field of gaming security and software engineering, highlighting the need for informed development practices that carefully consider the intersection of effective anti-cheat mechanisms and user privacy.

CCS CONCEPTS

• **Security and privacy** → *Software reverse engineering; Software security engineering; Systems security.*

KEYWORDS

kernel-level anti-cheat, operating system security, privacy, rootkit characteristics, software intrusiveness

ACM Reference Format:

Christoph Dorner and Lukas Daniel Klausner. 2024. If It Looks Like a Rootkit and Deceives Like a Rootkit: A Critical Examination of Kernel-Level Anti-Cheat Systems. In *The 19th International Conference on Availability, Reliability and Security (ARES 2024)*, July 30–August 02, 2024, Vienna, Austria. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3664476.3670433>

1 INTRODUCTION

Online video gaming, particularly competitive gaming and esports, consistently has to confront the issue of cheating. This challenge has intensified with technological advancements in gaming, prompting a response from game developers in the form of sophisticated

anti-cheat systems. Notably, kernel-level anti-cheat systems have emerged as a solution to this issue; such systems operate at a deep system level, a characteristic they share with rootkits. Rootkits, known for stealth and deep system integration, typically serve unauthorised purposes, contrasting with the intended protective role of anti-cheat systems.

This paper examines the similarities and differences between kernel-level anti-cheat systems and rootkits. It begins by outlining the progression of cheating methods in online gaming and the corresponding evolution of anti-cheat technologies, focussing on those operating at the kernel level. The discussion then shifts to rootkits, defining their key characteristics for comparison. The primary analysis involves applying a set of metrics, developed for classifying rootkits, to various kernel-level anti-cheat systems used in current popular video games. This methodology is employed to assess whether these anti-cheat systems exhibit rootkit-like properties, potentially raising concerns about user privacy and system integrity.

The aim of this paper is to investigate the ethical and technical implications of kernel-level anti-cheat systems in online gaming. It seeks to address the balance between effective cheating prevention and safeguarding user privacy, a critical issue in the landscape of digital games. Our contributions are as follows:

- We propose a set of criteria to characterise the rootkit-ness of a piece of software.
- We analyse the most prevalent kernel-level anti-cheat systems currently in use in online games and evaluate them based on these criteria, finding two of the four to be rootkit-like.
- Finally, we discuss how to balance security and privacy concerns when designing and employing kernel-level anti-cheat software in digital games.

The rest of this paper is structured as follows: First, we briefly discuss the limited existing literature on the topic. We then give a short background on rootkit and lay out the metrics for rootkit-like behaviour we have synthesised from the field of rootkit research. Thereafter, we discuss, analyse and evaluate the four most prevalent kernel-level anti-cheat systems used in online gaming (BattlEye, Easy Anti-Cheat, FACEIT Anti-Cheat and Vanguard) against these metrics, finding FACEIT Anti-Cheat and Vanguard to exhibit clear rootkit-like behaviour. Finally, we summarise our findings and discuss their implications, in particular with regard to taking user privacy into account when designing anti-cheat software.

This article thus contributes to the existing body of knowledge by providing insight into the methods utilised by kernel-level anti-cheat systems, classifying them as rootkits or non-rootkits based on their intrusiveness. The findings serve as a starting point for further



This work is licensed under a [Creative Commons Attribution International 4.0 License](https://creativecommons.org/licenses/by/4.0/).

ARES 2024, July 30–August 02, 2024, Vienna, Austria
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1718-5/24/07
<https://doi.org/10.1145/3664476.3670433>

investigations and discussion on the delicate balance between cheat detection and the preservation of user privacy and system integrity within the gaming community.

2 RELATED WORK

Although there are numerous rootkits and kernel-level anti-cheat solutions, there has thus far been very little research focussing on examining and contrasting these two types of software. While several papers have been published discussing rootkits (which we discuss in more detail in [section 3](#)), their detection methods and typical behaviours, there is a noticeable lack of research on anti-cheat systems, with the limited existing literature thereon (much of it in the form of bachelor's and master's theses) often lacking in-depth analysis and comprehensive evaluation.

Rendenbach [25] defines a basic framework for game developers to discern which kinds of anti-cheat system best meet the demands of the game. This framework includes a number of metrics, such as the ethical stance on cheating of the developers, the available budget, privacy concerns and other relevant factors. By considering these metrics, game developers can make informed decisions on implementing an anti-cheat system that aligns with the specific requirements of the game and their ethical considerations. Following this framework, the author provides an overview of possible options that encompasses existing anti-cheat systems as well as other technical and psychological options. This comprehensive overview explores the various measures available and their effectiveness in addressing cheating in games. The discussion concludes with the proposal of a “trustworthiness” factor, which combines the presented measures to assess the overall reliability and efficacy of an anti-cheat system.

Lehtonen [14] provides an overview of the historical development of anti-cheat systems in video games. The study categorises and examines different techniques employed by anti-cheat systems. Additionally, the research evaluates and describes the effectiveness of combined approaches utilised by these systems. By presenting this comprehensive analysis, Lehtonen contributes to understanding anti-cheat systems in video games, their evolution and the evaluation of combined approaches for enhancing gameplay integrity and fair competition.

Silva [29] elaborates on anti-cheat methods solely residing on the server to increase users' privacy. Due to many games having the ability to replay matches, these demos can be analysed in retrospect, detecting any occurrences of cheating in the match.

Maario et al. [17] propose the need for less intrusive anti-cheat systems and introduce an approach for such a system. In the paper, several types of cheats and anti-cheat measures are elaborated. Mirroring Silva, the conclusion of this paper also favours a server-sided anti-cheat mechanism utilising recorded games and evaluating them based on machine-learning approaches.

Beagle [1] presents an overview of the evolution of rootkits over time and gives a detailed classification of virtualised rootkits, categorising them based on specific characteristics and behaviours. Additionally, the study addresses the resulting threats to information security, covering prevention strategies, detection methods

and reliable removal techniques. The study thus provides a comprehensive understanding of rootkits, their various types and the associated challenges in ensuring robust information security.

Jiang's work [12] introduces a virtualised honeypot farm to attract and capture rootkit attacks to obtain samples from real-world incidents. This honeypot farm collects malware samples from the internet for subsequent analysis. The study also discusses how to design a highly efficient and accurate malware logger system. This logger system is aimed at effectively recording and analysing malware activities to enhance the understanding and response to such threats. By combining the virtualised honeypot farm and the malware logger, Jiang's research contributes to the field of cybersecurity by providing valuable insights into rootkit attacks and subsequent prevention methods and facilitating the analysis of captured malware samples. However, not all rootkits would execute on the virtualised platform.

3 METHODOLOGY

In response to growing concerns within the online gaming community about the similarities between kernel-level anti-cheat systems and rootkits, this study employs a structured analytical approach. We focus on comparing prominent kernel-level anti-cheat systems against defined metrics associated with rootkits. The list of metrics used in this study is derived from an analysis of different studies concerning the behaviour of common rootkits, while the evaluation of the different anti-cheat solutions is based on empirical analyses of both security researchers and cheat developers. This approach enables us to portray the systems from different perspectives. An anti-cheat solution was only flagged as exhibiting a certain characteristic if clear evidence of rootkit-like behaviour was present. We selected the four kernel-level anti-cheat solutions analysed for analysis which are currently most prevalent and widely used in the online gaming community.

3.1 Rootkits

Rootkits are a type of malware that provide cybercriminals with unauthorised access to a computer system while remaining hidden. They are classifiable into several types: firmware rootkits, kernel-mode rootkits, virtualised rootkits and user-mode rootkits (although these ones are not relevant to this study).

Firmware rootkits [15] target the software that runs particular hardware components, such as the BIOS, and are stored on the software that runs during the boot process before the operating system (OS) starts. They are stealthy as they can persist through OS reinstallation because the firmware on the hardware itself is infected.

Kernel-mode rootkits [13] modify the OS by adding or changing existing code. They often exploit the fact that OSes allow device drivers or loadable modules to execute with the same privileges as the OS kernel. To this end, they are often packaged as device drivers or modules to avoid detection.

Virtualised rootkits [8, p. 9] boot up before the operating system and operate as malware that executes as a hypervisor controlling one or multiple virtual machines (VMs). This shim

between hardware and OS enables the rootkit to conduct its malicious activities with less chance of being detected since all VMs linked to the hypervisor appear to function normally.

3.2 Rootkit Metrics

After this brief description of different rootkit types, we now define a set of metrics to classify software as rootkit-like. We have synthesised a set of seven properties commonly attributed to rootkits in our classification scheme.

Evasion: Evasion and deception of intrusion prevention systems or standard antivirus scanners is a known feature of rootkits. Rootkits often accomplish this goal by hiding malicious processes or tampering with applications on the infected system, patching the binaries so they fail to show files created by the rootkit, and often employing hooking tactics [16]. If the rootkit starts before the OS has finished booting (as is the case in firmware rootkits), it can evade all detection mechanisms of the OS altogether due to it having control of all drivers and applications starting after itself [7]. In addition, rootkits often attempt to disguise themselves as a different process using process hollowing [20, p. 28]. In this strategy, the memory of harmless processes is deallocated and overwritten by the rootkit for its usage while still operating under the name of the harmless process. Some rootkits can also hide the generated network traffic amongst other features by manipulating the `ioctl` syscall, typically used to read or change the driver parameters for attached devices, including the network card [2, p. 244 ff.].

Virtualisation: Rootkits often use virtualisation for evasion tactics and concealment of their code. Virtualisation is used to protect the image itself by using proprietary, custom architecture emulations. Often tools like *VMProtect*¹ are used to protect software images, making it difficult to reverse-engineer such software. Despite the use of these measures, it is possible to (partially) dump such modules from loaded memory areas or with the use of third-party tools to de-virtualise the images, enabling reverse-engineering them nonetheless [27]. Despite sharing the same terminology, this kind of virtualisation is different from what is used in virtualised rootkits.

Time of execution: Firmware rootkits are designed to start execution concurrently with or before the OS of the infected system, depending on the methods utilised by the rootkit. Booting this early leads to the advantage of hiding from user-level monitoring tools and the OS's security mechanisms. Additionally, this leads to the malicious software being able to eavesdrop on every action happening above it by hooking various methods, using syscalls, and manipulating loaded libraries. With these capabilities, the rootkit can monitor any function executed in the user and kernel space, including alterations [19, p. 52 f.].

Remote access and controllability: Most rootkits implement methods for the threat actor to control and access the infected system remotely and often include infected targets in botnets. Infected machines in such networks become bots. Botnets are rentable to run large distributed denial-of-service attacks on various targets. Bots connect to a centralised command-and-control (C2) server infrastructure, which sends commands to the infected systems that the bot executes. Furthermore, an attacker can install additional malware onto the infected system using the remote access feature

and disable or remove security features, further compromising the infected systems [19, p. 319]. This feature is often used to stage supplementary attacks on adjacent systems to perform lateral movement in the network.

Information exfiltration: Rootkits often include functionalities to exfiltrate data from infected systems, especially targeting corporations or governments to steal classified or high-value information. Depending on the information stolen, it can be sold (e. g. on the dark web) or used for the attackers' gain. These techniques can also be used to obtain basic information about the system or the attached network to plan for further attacks and select suitable malware specifically targeting the infected systems' weaknesses [18, p. 80 f.].

Network manipulation: Given that the network card of a system requires drivers to operate, a rootkit can attach to these drivers by hooking functions or the syscall table directly to send and receive packets on the network. This technique enables the rootkit to stay undetected for user-mode applications and spoof the outgoing network connections, hiding the ones originating from the rootkit. Furthermore, monitoring and alteration of established connections are possible, often used to attack systems in the adjacent network or connect to a C2 server. This is especially useful for passively eavesdropping on the network.

Removability: Kernel-level rootkits can often be hard to remove, requiring specifically designed tools. It is even possible that the system needs to be reinstalled from scratch to remove all traces of the rootkit, which is the generally accepted best-practice approach to treat infected systems [1]. In some cases, replacing some or all system hardware components is necessary to remove a firmware rootkit entirely.

3.3 Anti-Cheat Solutions

The selection of kernel-level anti-cheat systems was based on those used in online games with the highest player counts (as recorded on steamcharts.com and tracker.gg on 21 January 2023). Additionally, the level of intrusiveness was considered, determined by the extent of controversy surrounding their functionality within the gaming community, as reflected in discussions on online forums². This resulted in the anti-cheat systems of the games *Counter-Strike: Global Offensive*, *Fortnite*, *Tom Clancy's Rainbow Six Siege* and *Valorant* being chosen. All four systems are partially based in kernel mode and use similar methods. Due to the intrusiveness of modern kernel-level anti-cheat solutions, the boundaries between them and rootkits are often slim, causing a rogue kernel-level anti-cheat system to have the potential to devastate the systems it is installed on. We have chosen a simple point system for our evaluation, with a single point added to the rootkit-ness score of an anti-cheat system for each matching criterion. Summing these points after the analysis, we arrive at a score from zero to seven; if any given anti-cheat system is assessed with four or more points, we consider it to be a rootkit-like system.

BattlEye. This German anti-cheat solution is well established in the gaming industry and used by various games from different genres, including *Rainbow Six Siege*, *PUBG: Battlegrounds* and others.

¹ <https://vmpsoft.com/>

² <https://www.reddit.com/r/pcmasterrace/comments/19acyup>

The self-proclaimed “gold standard” of anti-cheat solutions must be installed on the server hosting the game and on all clients playing it, giving it an in-depth overview and control of all in-game events. The protection mechanisms of BattlEye are circumventable relatively easily. Thus, many sources reveal this anti-cheat solution’s code and inner workings. BattlEye employs an account-based recognition system, which can issue global bans to rid a game of cheaters using multiple accounts.³

Easy Anti-Cheat. Easy Anti-Cheat (EAC) is more sophisticated than BattlEye, which makes it harder to reverse-engineer and discover the mechanisms of this solution. This anti-cheat is owned and developed by Epic Games and built into games like *Apex Legends*, *Fortnite* or *Rust*, amongst many others. EAC is free to use and easy to implement in any game for developers, which explains the popularity gained in recent years. Using the hardware ID (HWID) of the system, EAC can ban cheaters globally, increasing the difficulty for players with an unfair advantage to create new accounts and continue their cheating spree.

FACEIT Anti-Cheat. FACEIT Anti-Cheat is a third-party platform for playing games with a custom ranking system, often considered superior to those built into the games. *Counter-Strike: Global Offensive* is one of the provided games, serving up to 22 million players. It is considered a rather sophisticated anti-cheat system by other reverse engineers in the field⁴ and bypassing the security measures it implements is an exceedingly challenging task.⁵

Vanguard. The anti-cheat software Vanguard, developed by Riot Games, is a more intrusive solution. To ensure the successful launch of the protected games, the system must start in a secured state, necessitating the kernel driver to initialise during the boot process. Without this initialisation, the protected games fail to launch. At present, Vanguard is exclusively utilised for the game *Valorant*, but there are plans to extend its usage to include other games developed by the studio in the future [36].

4 RESULTS

We now turn to analysing anti-cheat solutions for their rootkit-ness based on the metrics defined in subsection 3.2. This involves a detailed examination of four anti-cheat solutions, resulting in their classification based on these metrics.

4.1 BattlEye

BattlEye consists of four components [33], working together to detect and ban cheaters:

BEService. BEService is part of the Infrastructure that enables communication to the BattlEye server from the user- and kernel-mode parts installed on the client.

BEDaisy. BEDaisy is the kernel-mode driver, allowing the anti-cheat system to enumerate the memory, scan for vulnerable libraries or insert other modules streamed from the server.

BEClient. BEClient is the user-mode part of the anti-cheat system, injected into the game upon startup, and issues most calls to the driver to execute the various scans.

BEServer. BEServer must be running on the game server the users connect to, where it can monitor user behaviour, receive all notifications from the BattlEye clients (through the BEClient), and issue bans for either the specific server or even globally.

These four parts constitute the base of the BattlEye anti-cheat system. Game developers can use the API for the modules in the game code, embedding it deep into the game. This yields monitoring results for in-game player behaviour, making the detection of blatant cheats easy. It interfaces with the kernel driver BEDaisy, which can observe low-level memory areas, check for existing function hooks, etc. To evade tampering from cheats operating at the user level, the anti-cheat system performs scans within the kernel. The client gets the modules streamed from the BEServer, inserting them directly into the memory where they are allocated and executed. Reporting players for any violation is implemented in the BEService part of BattlEye via the function `battleye::report()`, which encapsulates information about the player and the infringement conducted. When a player gets reported, it does not mean a ban is immediately issued. However, more scanning modules will be streamed to the client, gathering additional information about the system. BattlEye can hence be characterised as a relatively lenient anti-cheat system. It primarily issues outright bans only when players are caught cheating by deliberately evading scans and protection measures. The usage of e. g. macros will, at most, lead to being kicked out of the current game or the game not starting.⁶

4.1.1 BattlEye Anti-Cheat Measures: The components of BattlEye use the named pipe

`.\namedpipe\BattlEye` for inter-process communication, encrypting the traffic sent with a XOR encryption using weak keys, which makes it vulnerable for known-plaintext attacks. This is the primary communication method between the components BEDaisy, BEService and BEClient.

Memory Scans. To detect cheats and anomalies in user behaviour, the kernel driver scans the whole memory area of the protected game and the anti-cheat system, allowing it to check for known cheats loaded. Additionally, all external memory pages are examined to determine whether they have the X (execute) bit set. This implies that any process with sufficient privileges can execute the code residing in these pages. These pages can be utilised to inject cheats or a loader into the memory, which subsequently can be used to exploit vulnerabilities in the game or circumvent the anti-cheat system [33]. To obstruct cheat developers from using such memory areas, several checks are put in place by BattlEye through memory scans: anomaly scans, pattern scans and module-specific scans.

Anomaly scans are used to detect areas of memory with address offsets that are not associated with any known loaded image, meaning all injected libraries or other tampering attempts with the memory of the game. In case unknown memory areas are detected, BattlEye sends a report with information concerning the violated memory areas.

³ <https://www.battleye.com/about/>

⁴ <https://guidedhacking.com/threads/anticheat-faceit-bypass.16113/>

⁵ <https://www.faceit.com/en>

⁶ <https://www.battleye.com/support/faq/>

Pattern scans use basic antivirus tactics, comparing loaded modules with known signatures of cheats or methods of injection. Some of these signatures are hardcoded into the BEClient, but most are streamed from the BEServer to the BEClient. BattlEye iterates over the whole game memory and the flagged pages, comparing the streamed signatures with the identified ones. If a known pattern is detected, a report is issued.

Module-specific scans check the memory for any modules that can be used to tamper with the memory of the game. These modules contain checks for specific DLLs, including `mmres.dll` and `mshtml.dll`. Other specific module identifiers, based on in-memory offsets and timestamps, are streamed from the BEServer during the execution of the game, which are most likely used to detect other libraries used for cheat injection [33]. Additional scans are conducted to inspect other loaded modules, particularly kernel modules. These modules are enumerated by invoking the function `NtQuerySystemInformation()`, which provides comprehensive details about all the modules loaded into the system. If a module found in the blocklist is detected within the kernel, such as a known vulnerable driver that has been exploited to inject malicious code into the kernel, BattlEye takes immediate action, terminating the game and transmitting a report to the BEServer [22].

Page Guard. BattlEye safeguards the memory areas of the game by inspecting memory pages for the presence of the `PAGE_GUARD` flag. This flag is a memory protection option that can be enabled when allocating or protecting a page in memory. Once set, it designates a guard page that triggers a `STATUS_GUARD_PAGE` exception when accessed, alerting BattlEye to potential unauthorised or malicious activity targeting the protected memory region. Utilising the `PAGE_GUARD` flag is beneficial in detecting memory corruption problems or identifying malicious code attempting to write into protected memory regions.⁷ However, it appears that BattlEye does not rely on this feature in practice, only checking if the protection of the requested memory page was changed [33].

Window Enumeration. To detect hooking and, in turn, spoofing of functions in the BattlEye modules, all visible windows on the desktop are enumerated. Overlapping windows are iterated, starting with the highest z-index (corresponding to the uppermost window) and decrementing the index for each window, getting the window handle according to the current one. In case fewer than two windows are detected by this method, the anti-cheat assumes its functions were hooked and returned fabricated results, leading to a report [33].

Process Enumeration. On each occasion this module is invoked, the anti-cheat system snapshots all currently running processes, checking each for various anomalies. Based on a blocklist of applications commonly used to inject cheats, when such a process is found, BattlEye will report the user [33]. Apart from scanning for process names, BattlEye also verifies the path of the running process.

LSASS Checks. The Local Security Authority Subsystem Service (LSASS) validates credentials when a user attempts to log in on a Microsoft Windows OS, enforcing the local security policy and issuing security tokens.⁸ Historically, the Local Security Authority (LSA) has been targeted for infecting systems with viruses, taking advantage of its vulnerabilities. Due to the responsibilities of the LSA, BattlEye not only checks but hooks the syscalls issued from and to it, redirecting those calls to the BEDaisy driver, which can monitor every process from the kernel and crash the system when an invalid call is issued [33].

Networking. BattlEye conducts system scans to detect all open TCP connections to known cheating sites. These sites are utilised for DRM purposes, which help address the issue of illegal software distribution faced by cheat developers and regular game developers alike. When starting the cheat, it must endure such a DRM check by connecting to a server and authenticating the user with valid credentials, ensuring the user paid for the cheats. This connection must often be kept established when the cheat is in use. Open TCP connections are listed in the TCP table of the system and thus are easily readable for the anti-cheat system. A report is generated if an IP address of a known cheating provider is found [33].

Tick Scan. If this test is invoked, BattlEye will schedule the current thread to sleep for exactly one second. Before the sleep command is issued, it obtains the current (game) tick count and compares it with the tick count after sleeping for the second. If the tick delta is more than 1200 ms, it will generate a report [33]. This check attempts to detect if an attacker injected code to speed up the game ticks, which could indicate a speed hack, i.e. the player can move faster in-game than intended. Another way this check can be used is to detect if the game is executed inside a virtual machine, which often has more overhead and thus takes longer to execute the same instructions than bare metal [34].

Other Methods. There are other checks this anti-cheat system conducts, mostly scans for applications like 7zip or Visual Studio Code. Additionally, BattlEye regularly transmits information to the server, which includes details about all processes, particularly those with active handles on the game. It also provides game file sizes for integrity checks and reports information regarding hooks detected on any process [33]. If the BEServer is not certain if a player is cheating, it requests more information by streaming the shellcode to the BEClient, which sends back all information, leading to substantial information exfiltration.

4.1.2 BattlEye Self-Protection. BattlEye not only safeguards the intended game but also protects itself. It extends the established practices outlined in subsection 4.1.1 by employing binary virtualisation and integrity checks. The virtualisation of the images is conducted with *VMProtect*, hardening the reverse-engineering and disassembling process. The integrity checks are performed by searching for the file in memory, acquiring a handle and subsequently attempting to open the file. This approach allows BattlEye to verify the integrity of the file by confirming its presence in memory and ensuring that it can be successfully accessed. If this

⁷ <https://learn.microsoft.com/en-us/windows/win32/memory/memory-protection-constants>

⁸ <https://learn.microsoft.com/en-us/windows-server/security/windows-authentication/credentials-processes-in-windows-authentication>

operation is successful, BattlEye will get the certificate of the file and compare it against a list of prohibited certificates. In the event of an attempt to load a forbidden file, the anti-cheat system notifies the user regarding the tampering and prohibits the game from loading [38].

Uninstalling. Uninstalling the BattlEye anti-cheat system varies for each game, but generally it will uninstall with the game. If not, it can be removed with the script `Uninstall_BattlEye.bat`, which is installed along with BattlEye. To remove the service created when installing BattlEye, the command `sc delete BEService` must be executed from an elevated command prompt.⁹

4.2 Easy Anti-Cheat

EAC is the anti-cheat solution developed by Epic Games, obtained by the company in 2018. It is provided free to use for game developers.¹⁰ Well-known games using EAC, amongst others, are *Apex Legends*, *Dead by Daylight* or *Fortnite*. When assessing the security features of EAC, it is considerably more challenging to obtain the reverse-engineered source code compared to BattlEye, due to the increased security measures in the software.

EAC is designed to implement global bans for players who engage in misconduct by banning their corresponding HWID. This prevents cheaters from creating new accounts and accessing the game from the same machine. In contrast to BattlEye, global bans through EAC are restricted to the game the player has cheated in, not extending to other games using EAC. The HWID used to identify the systems of the players is generated from the following identifiers:

Registry Keys. Keys containing system hardware identifiers, information about the BIOS, the installed graphics card, and others [24].

MAC Address. EAC scans the system for all network interface cards (NICs) using their MAC address in the HWID.¹¹

Disk Serial Numbers. Every single disk possesses a unique serial number to identify it. EAC incorporates the unique serial numbers of all installed disks in the system as part of the HWID.¹¹

WMI Queries. The Windows Management Instrumentation (WMI) is used by EAC to query specific identifiers from the actual hardware via the `Win32_BaseBoard` class. EAC utilises identifiers from the motherboard and the GPU.¹¹

Processor Features. CPUs do not have unique identifiers, just the model number and the given feature set. These metrics are included in the generation of the HWID.¹¹

While the HWID generation incorporates the metrics mentioned earlier (and possibly others), it is still technically feasible to spoof all of these values. However, this requires significant effort and expertise to do successfully. This system is hence not flawless, but given the dedication it takes to fabricate these values and the secrecy about them from the official publishers, it is deemed good enough to issue effective global bans.

4.2.1 EAC Anti-Cheat Measures. With an understanding of the generation of the HWID used for banning players with an unfair advantage, it is possible to delve into the mechanisms EAC employs to detect cheating players. EAC consists of several parts cooperating to protect the game against cheaters. Each time a game protected by EAC is launched, it initiates the download of an up-to-date package that includes the driver module and the game-specific module from the EAC content delivery network. This ensures that the necessary components for EAC's operation, including the latest driver and game module versions, are obtained before the game is executed. Meanwhile, the EAC executable is launched, opening a shared buffer that allows communication between the kernel driver and the user-mode application. The kernel driver is used to monitor the system for various malicious changes. Furthermore, the `EasyAntiCheat.dll` is injected into the game, enabling EAC to inspect all changes and input registered in the game [3]. The methods employed by EAC to detect cheaters are the following:

Hook Detection and Blocking. The kernel driver of EAC can detect various hooking techniques, alerting the anti-cheat system when such a hook is detected outside the memory of the game. If a malicious hook is identified, the kernel driver will automatically block this hook and communicate via the shared buffer to generate a report. The purpose of implementing these measures is to impede cheaters from injecting malicious code into the game and to prevent the creation of process handles that could be exploited by external tools to interact with the game [24].

Memory Scans. Like BattlEye, EAC scans the memory of the game, including adjacent memory areas, to detect loaded cheats. The techniques used are very similar to those BattlEye employs [24].

Driver Scanning and Logging. The EAC kernel driver performs scans on the system to identify and block any loaded drivers that may be considered malicious. Specific known drivers it scans for are `Dbgv.sys` and `PROCMON23.sys`, two drivers of the Sysinternals Suite, as well as `dbk64.sys`, the driver for the tool Cheat Engine.

In addition to scanning and blocking suspicious drivers, EAC generates detailed logs that capture information about loaded drivers and loading attempts within the system. These logs are forwarded from the kernel driver to the service executed in user mode, encrypted, and then sent to the EAC servers.

Stack Walking. EAC applies stack walking to enhance hook detection, a technique to trace the origin of a method call in memory. Using the two functions `RtlLookupFunctionEntry()` and `RtlVirtualUnwind()`, it is possible to obtain the base address of the image hooking the application [24].

Hypervisor Detection. EAC uses the feature set of CPUs to detect if it is executed in a virtualised environment or on an OS installed on bare metal, accomplished by executing a single `vmread` instruction upon starting. This instruction attempts to read a field from the virtual machine control structure [10, p. 2491]. By verifying the successful execution of the `vmread` instruction without triggering an exception, EAC can determine if it is running inside a virtual machine. If the instruction executes without issues, EAC blocks the game from launching [6]. In addition to the VM test, EAC employs

⁹ <https://www.battleye.com/support/faq/>

¹⁰ <https://www.easy.ac/licensing/>

¹¹ <https://github.com/ch4ncellor/EAC-Reversal/blob/main/hwid.cpp>

the same tactic as BattlEye, sleeping for one second and measuring the game ticks happening in between, as already described in [section 4.1.1](#) [6].

Window Enumeration. Like BattlEye (see [subsection 4.1.1](#)), EAC employs a technique to enumerate all visible windows during runtime, obtaining the handle and the window name. If a window is detected containing a name corresponding to any cheat, a report is generated [24].

Manual Mapping Detection. Manual mapping describes loading a DLL into memory using custom tools instead of Windows API functions. This technique offers the benefit of total control over the location where the library is loaded, circumventing automatic detections from both the OS and the applications involved [4]. EAC employs a combined approach of memory scanning to identify manually mapped modules or drivers. The routine begins with scanning processes and threads, during which the associated image bases are saved. Subsequently, the captured base addresses in the virtual memory space are examined for anomalies, such as Portable Executable headers indicating the presence of injected applications within the memory area. Furthermore, like BattlEye, the system verifies adjacent memory regions with the execute bit set, actively searching for any non-module code present [24]. If any modules are detected in the areas outside of the game not associated with a module, the anti-cheat system will dump all strings and syscall invocations found in the section and send them to the developers in an encrypted form.

Handles and Threads. EAC obtains a list of all open handles on the system, using them to search for any process accessing the game without authorisation. If such a process can be located, the user is flagged. Furthermore, handles pointing to sections in the physical memory are probed to check if any process writes code directly into memory which could be executed in the context of the game or the anti-cheat system [24]. Additionally, EAC scans for all threads running in the kernel or user mode and compares them with defined detections for suspicious threads. If those threads are called from an executable memory region not associated with any loaded module of the EAC system, it generates a report [24].

Instrumentation Callbacks. Instrumentation callbacks enable simple syscall monitors from user mode in EAC. Whenever a syscall is executed and the kernel returns to user mode, the anti-cheat system checks for the address of the `InstrumentationCallback`, which is stored within the `KPROCESS` structure. If this address points to a valid memory space, the kernel swaps the return address of the function for the one inside the `InstrumentationCallback` member. The substitution of the return address during the transition from kernel to user mode in EAC occurs only when the `dr7` register is set, triggering a detour. Without setting this register, the return address remains unchanged, and the transition from kernel to user mode proceeds normally without any modifications. When a user executes a game with EAC enabled, the kernel driver creates an instrumentation callback routine that monitors the system calls and other events made from the game.¹² In this callback function,

EAC can scan for any cheating attempts by using techniques like stack walking to find the module that executed the syscall in the first place.

Game Integrity Checks. During runtime, EAC employs integrity validation by downloading a catalogue of valid signatures encompassing all game files. This catalogue is used to verify the integrity of game files whenever the game accesses them. Furthermore, the configuration containing the definitions of files to probe is also signed and checked at runtime so that a malicious actor cannot tamper with it. If files experience prohibited modifications mismatching these signatures, EAC restricts the user from playing online games, but allows offline gaming.¹³ In addition to its existing detection mechanisms, the EAC driver incorporates checks that prevent the simultaneous execution of games protected by EAC and those protected by BattlEye. Specifically, if the BattlEye kernel driver `BEDaisy.sys` (as described in [subsection 4.1](#)) is detected to be running on the system, the game protected by EAC will not start [28].

4.2.2 Easy Anti-Cheat Self-Protection. Like BattlEye, EAC employs self-protecting techniques as well. There is little public knowledge about this topic, but it is generally accepted that these measures are similar to those of BattlEye. EAC also utilises a proprietary virtual machine obfuscator, which conceals the binaries and executes them on non-standard architecture. In order to perform self-checks, EAC employs the use of the virtual `vmcall` instruction. This instruction allows EAC to compare the loaded driver in memory with a clean copy that was read from the disk before the memory mapping process. If the anti-cheat system detects any discrepancies in the integrity of the code, it deliberately crashes the protected application by attempting to access the invalid memory area `0xEAC` [11, 30].

Uninstalling. Uninstalling Easy Anti-Cheat from the system happens automatically when the last game using this anti-cheat solution is uninstalled. If remnants remain, EAC can be manually uninstalled by running the installer and selecting the uninstall option. This rids the system of all residues of EAC.¹⁴

4.3 FACEIT Anti-Cheat

FACEIT Anti-Cheat is a third-party service for the games *Counter-Strike: Global Offensive*, *Dota 2* and *Team Fortress 2*, amongst others. It offers ranked leagues with better anti-cheat capabilities than the built-in matchmaking systems in the mentioned games. FACEIT Anti-Cheat uses a combined approach to detect cheating players by collecting data from the locally installed FACEIT Anti-Cheat client and behavioural analysis of players recorded on the server. The latter system is based on the FACEIT Anti-Cheat servers and thus will not be analysed in this article. This approach allows for comparing multiple data sources, making it harder for players to cheat in such matches.

FACEIT Anti-Cheat heavily restricts the usage of the anti-cheat client with regard to the platform the game is played on and bans players for attempts to circumvent these measures, considering them cheating. The terms of service of FACEIT Anti-Cheat explicitly prohibit the installation of the client, including the anti-cheat

¹² <https://www.unknowncheats.me/forum/anti-cheat-bypass/561479-eacs-instrumentation-callback-bypass.html>

¹³ <https://dev.epicgames.com/docs/game-services/anti-cheat/using-anti-cheat>

¹⁴ <https://www.easy.ac/en-us/support/game/guides/installer/>

system, on a virtual machine. Engaging in such behaviour is considered a violation resulting in a ban from the platform. The attempt to debug or decompile any part of the anti-cheat solution is also considered cheating.¹⁵ FACEIT Anti-Cheat put these measures in place to protect the anti-cheat from malicious individuals attempting to detect exploits.

4.3.1 FACEIT Anti-Cheat Measures. Consisting of a kernel driver (FACEIT.sys), the FACEIT Anti-Cheat client and the server-side evaluation, FACEIT Anti-Cheat offers a sophisticated framework to protect the offered games against cheaters. The kernel driver utilises similar anti-cheat tactics as previously discussed anti-cheat systems.¹⁶ The FACEIT Anti-Cheat driver boots with the operating system to ensure the system is in a trusted state. The FACEIT Anti-Cheat kernel driver is designed to constantly collect data about the system, including other running drivers and executed programs, transmitting it to the server when the client starts to detect any illegal behaviour indicators. Known FACEIT Anti-Cheat tactics on the client and the server side are as follows:

Instrumentation Callbacks. FACEIT Anti-Cheat places instrumentation callbacks on several Windows kernel functions. The anti-cheat system blocks the loading of external modules and DLLs, logs drivers and general loading events via a callback on the LoadImage() function. Further callbacks are placed on the CreateProcess() and CreateThread() functions, including all variations, which notify the anti-cheat system about created processes or threads for logging and reporting [23].

Driver Detection and Stripping. The anti-cheat system has built-in detection and unloading capabilities for vulnerable drivers that could inject malicious code for cheats (similar to Vanguard, see subsection 4.4.1). Additionally, mechanisms are in place to locate and close open handles to the protected game to ensure no other application is interfering [23].

Virtualisation Detection. Due to the strict approach against installation on virtual machines, a detection mechanism against this is in place. FACEIT Anti-Cheat blocks execution with the Windows built-in virtualisation solution Hyper-V enabled, requiring the players to execute custom commands to fully disable it. According to FACEIT, it is impossible to “maintain a fair playing environment” with this feature enabled.¹⁷ They go so far as to instruct players to disable the memory integrity feature of the Windows kernel (which considerably weakens the system by dropping the special protection for high-security processes in memory) so they can better monitor processes like LSASS and CSRSS.

Virtualised Binaries. FACEIT Anti-Cheat and the client utilise VMProtect for binary virtualisation and commercial packers for further obfuscation, similar to all the previous anti-cheat systems [5].

System Security. The FACEIT Anti-Cheat system requires the active use of a Trusted Platform Module to store keys to encrypt

the data sent to the servers.¹⁸ Furthermore, it is necessary to enable the Data Execution Prevention feature to enhance protection in memory, so pages cannot be written to and executed at the same time, blocking tactics to inject cheats into the memory.¹⁹ In addition to those requirements, FACEIT Anti-Cheat needs the system to be booted with Secure Boot ensuring the integrity of the system beginning with the UEFI. This often requires changes to the UEFI and reformatting of system disks initialised with MBR instead of GPT.²⁰

Due to FACEIT Anti-Cheat being very secretive about the anti-cheat system and discouraging system analysis, very little data is available. Due to this, it can be assumed that there are additional detection and monitoring features included in the anti-cheat system which are not covered above. If players are detected to be cheating, the system flags them for the future, but does not issue instant bans to avoid letting cheaters know which specific action triggered the system. Such flagged accounts are typically banned within one week after the incident. To obstruct cheaters from creating new accounts, FACEIT Anti-Cheat uses HWID bans. However, it is worth noting that the HWID generation method used by FACEIT Anti-Cheat is considered weaker than other anti-cheat solutions like Vanguard, as discussed in subsection 4.4. Only two known values are included in the HWID of FACEIT Anti-Cheat: The MAC addresses of the local NICs and the serial number of the first disk in the system. Using these relatively simple indicators, the FACEIT Anti-Cheat kernel driver continuously monitors any changes and spoofing attempts made to those values and notifies the server when the client is executed.

Uninstalling. FACEIT Anti-Cheat offers no clear instructions for removing the client or the anti-cheat system. Both can be removed via the default Windows uninstalling menu, but there is no guarantee that no traces of the applications will remain on the system. Furthermore, no information is available to revert the changes needed to start the anti-cheat system in the first place, making the whole process very intransparent.

4.4 Vanguard

Vanguard, developed by Riot Games specifically for the game *Valorant*, is widely recognised as one of the most intrusive anti-cheat systems.²¹ It operates upon system boot, ensuring the system starts in a trusted state. Vanguard comprises three distinct components:

vgk.sys is the kernel driver of the anti-cheat system, creating a protected memory area for the user mode part of Vanguard and the game itself. It features user-mode and self-integrity checks and communicates with the user-mode part of Vanguard, enabling command execution from there, including remote update and uninstall methods [21].

vgc.exe is the user-mode counterpart to the kernel driver, issuing the commands to execute various scans. It ensures the integrity of the game and the reporting functionality to

¹⁵ <https://support.faceit.com/hc/en-us/articles/360015788779-What-is-deemed-to-be-a-cheat>

¹⁶ <https://support.faceit.com/hc/en-us/articles/9394666828188-What-is-FACEIT-Anti-cheat-and-how-does-it-work>

¹⁷ <https://support.faceit.com/hc/en-us/articles/360019809319-You-need-to-disable-Hyper-V-to-launch-FACEIT-AC>

¹⁸ <https://support.faceit.com/hc/en-us/articles/4407006362002-Error-Please-enable-TPM-2-0-to-continue>

¹⁹ <https://support.faceit.com/hc/en-us/articles/360017192040-You-need-to-enable-DEP-to-launch-FACEIT-AC>

²⁰ <https://support.faceit.com/hc/en-us/articles/4406281700370-Secure-Boot-needs-to-be-enabled-to-launch-FACEIT-AC>

²¹ <https://www.gamechampions.com/en/blog/valorant-anti-cheat-vanguard/>

System	Evasion	Virtualisation	Time of Execution	Remote Access	Information Exfiltration	Network Manipulation	Removeability	Sum
BattlEye	○	●	○	●	●	○	○	3
Easy Anti-Cheat	○	●	○	○	●	○	○	2
FACET Anti-Cheat	●	●	●	○	●	○	●	5
Vanguard	●	●	●	○	●	○	○	4
Flame	●	○	○	●	●	●	●	5

Table 1: Overview of the analysed anti-cheat systems, including a comparison with the Flame rootkit.

the game servers. Direct communication with *Valorant* is also implemented [21].

Valorant.exe is the game protected by Vanguard. It is executed in the protected memory area created by the kernel driver and continuously checked by the user-mode anti-cheat part of the system [21].

Like the previous anti-cheat solutions, Vanguard utilises the HWID for player identification and banning. Riot Games has taken significant measures to protect the details and inner workings of Vanguard. Publicly available information about the anti-cheat system is limited due to the bug bounty program the company offers. While the specific details of Vanguard’s HWID generation process are not publicly disclosed, certain values are commonly assumed to be used in the generation of the HWID. These values, although not confirmed by Riot Games, are often speculated to include:

Disk Serials. Similar to EAC (see subsection 4.2), it is believed that Vanguard incorporates the serial numbers and volume identifiers of the disks present in the system as part of its HWID generation process [9].

Registry Keys. In case the registry key `HKLM\textbackslash SOFTWARE\Microsoft\textbackslash Windows NT\textbackslash CurrentVersion\textbackslash BuildGUIDEx` is available, it is speculated that Vanguard includes the value of this key in the HWID generation process. The `BuildGUIDEx` registry key typically contains a unique identifier associated with the specific build of the Windows OS installed [9].

NVRAM Variables. Vanguard reportedly incorporates specific information about the platform the system is running on in the HWID generation process. According to [9], this information is sourced from the `PlatformData` variable, which is said to include a collection of variables set by various devices installed in the system. However, it is important to note that the `PlatformData` variable itself is not defined in the official UEFI specification [31, p. 1739].

MAC Addresses. Like EAC, Vanguard is believed to use the MAC addresses of local NICs in the HWID.

Mainboard UUID. It is also considered plausible that Vanguard leverages the unique identifier stored in the BIOS or UEFI NVRAM of each mainboard during the HWID generation process [9].

Valorant is free-to-play, thus in principle easily allowing cheaters to create new accounts once the previous ones were banned for violations. This emphasises the need for HWID solutions because it is the sole indicator that can identify a system that was cheated on with any certainty.

4.4.1 Vanguard Anti-Cheat Measures. Vanguard employs several anti-cheat measures already discussed in detail for EAC, such as

instrumentation callbacks or various types of memory scans. To avoid needless repetition, we will focus on measures specific to Vanguard in this subsection.

Shadow Memory. Vanguard uses a sophisticated method to hide memory pages from other processes and threads executed on the system, protecting them from unauthorised access. This is implemented by hooking the `SwapContext()` ioctl function of the Windows kernel, called every time a context switch occurs. By hooking this function, denying access to threads not exclusively allowed is possible. Furthermore, Vanguard uses custom memory addresses in the code based on the shadow memory regions. A crash would occur if any other application attempted to access such a memory address except Vanguard, due to catching occurring page-faults within. Resulting from this refined approach, Vanguard is well protected against this kind of cheats by making it very difficult to manipulate the memory of the game [37].

Virtualisation. Vanguard and the associated binaries are virtualised and compressed with the proprietary software packer *Packman*, developed by Riot Games. The packer is utilised to obfuscate and encrypt the data. The essential functions of Vanguard are decrypted at launch, meaning that not all functions are loaded into memory simultaneously. This approach impedes attackers and security researchers from easily understanding and analysing the code [26].

Remote Controllability. Vanguard includes features that allow developers to remotely disable and uninstall the anti-cheat system in the event of discovering severe vulnerabilities [21]. If the feature that enables remote disabling and uninstallation of Vanguard were to be hijacked or exploited by attackers, it poses a severe security risk, granting attackers access to the kernel.

Limitations. To enhance the effectiveness of Vanguard, the game is designed not to start if the kernel driver of the anti-cheat system has not been loaded during the boot process. This approach establishes a secure and controlled environment where the anti-cheat system can operate without interference or circumvention attempts. By ensuring that the kernel driver is loaded before the game starts, *Valorant* can prevent other drivers or unauthorised software from being loaded beforehand. If any of those drivers must be loaded when the system is started in the trusted mode, it can be turned off from user mode. This results in the system falling into an untrusted state, which requires a reboot to re-enable the anti-cheat driver. *Valorant* can only be started when the system is in a trusted state.

Uninstalling. Uninstalling Vanguard is straightforward – searching for it in the installed applications and selecting uninstall. This removes the files of the anti-cheat system; a reboot is required to unload it from memory, as well [35].

4.5 Discussion

We now discuss the findings based on our analysis in [section 4](#). While no anti-cheat system shows signs of network manipulation, we discovered evidence for all other metrics in at least one of the systems analysed. A summary of the evaluation for the different anti-cheat systems against our metrics is shown in [Table 1](#), including a comparison with *Flame* [32], a rootkit detected by Kaspersky Lab in the Middle East in 2012.

In the category of evasion, Vanguard and FACEIT Anti-Cheat indicated significant traces. Vanguard employs some very intrusive tactics, including evasion tactics circumventing other threads and processes by implementing a custom memory management system, hiding from processes and threads, while FACEIT Anti-Cheat employs vigorous checks for virtualised environments, which is a tactic used by malware and rootkits alike to protect themselves against analysis. While this behaviour was introduced to stop cheaters from tampering with the virtual memory of the game, banning players for merely attempting to play in a VM could be considered unreasonable. Furthermore, the constraint to disable Hyper-V (blocking mundane programs like *WSL-2*) to ensure no virtualisation can occur at all strengthens the evasion tactics. These combined approaches each indicate rootkit-like behaviour.

All of the analysed anti-cheat systems tested positive in the category of virtualisation. All systems employ some kind of virtualisation, each in different ways. BattlEye uses *VMProtect* to virtualise the binaries. Similarly, EAC uses custom software to virtualise the binaries as described in [subsection 4.1.2](#) for self-protection. FACEIT Anti-Cheat employs heavy obfuscation tactics, as well. Lastly, Vanguard utilises a proprietary packer, including binary encryption by design, which hardens Vanguard against static and dynamic analysis. These methods make it challenging to reverse-engineer and assess them for any security concerns.

Similar to the category of evasion, FACEIT Anti-Cheat and Vanguard are the only two systems designed to execute when the system boots, leading to a positive result for the time of execution metric. FACEIT Anti-Cheat and Vanguard both require the kernel driver to boot with the operating system to check for any other vulnerable drivers to block, ensuring no cheats can be loaded on the system. Furthermore, FACEIT Anti-Cheat requires Secure Boot to be enabled, so no disallowed drivers or applications can be loaded before the OS starts. This design is the most intrusive of all anti-cheat systems. The other two kernel-level anti-cheat systems only start when the protected game is launched, thus posing no threat in this category.

Concerning remote access methods, traces could only be located in the binary of BattlEye, due to it sending a large amount of information to the BEServer backend. Although the other solutions show traces of such behaviour (e.g. Vanguard includes features for the developers to disable or uninstall the system remotely in case severe vulnerabilities are discovered), there is no concrete evidence of real remote access and controllability built into the other anti-cheat solution.

Regarding information exfiltration, all of the kernel-level anti-cheat systems indicated positive results for this metric. BattlEye sends a lot of information about the system and the game to the BEServer backend, while EAC utilises various system identifiers

to generate the HWID for identification purposes. Additionally, it logs all drivers and modules in memory and sends this information to the server for analysis and monitoring. This information is continuously transmitted to the server, regardless of whether a player is suspected of cheating. If this data, designed to identify a system uniquely, can be accessed by unauthorised people, tracking or impersonating victims on various game servers is possible. Both FACEIT Anti-Cheat and Vanguard employ reporting methods to send system information, such as the HWID, loaded drivers, modules and crash reports, to the game developers for analysis as well, furthermore heavily relying on analysing logs (including loaded drivers, process and thread events, etc.), which are sent from the client to the server.

Removing the anti-cheat solutions from the system is elementary for most analysed systems except FACEIT Anti-Cheat. Due to the lack of guides describing its proper removal, there is a chance that remnants of it remain active on the disk after removal attempts, thus possibly acting like a rootkit. Also, no additional guides are available for restoring the system to a state before using the software, leaving it in a possible vulnerable state, especially considering the requirement of disabling the memory integrity feature.

5 CONCLUSION

This article analyses the behaviour and tactics of four well-known kernel-level anti-cheat systems widely used in online gaming and compares them to functionalities and methods used in rootkits. The anti-cheat systems BattlEye and Easy Anti-Cheat showed minor similarities to rootkits, which were insufficient to classify them as such according to our metrics. FACEIT Anti-Cheat and Vanguard, however, were identified as rootkit-like applications due to the utilisation of comparable methods. The findings and insights from this research shed light on the thin line between what distinguishes kernel-level anti-cheat systems from rootkits.

The classification of anti-cheat systems as either rootkit-like or not presents significant challenges. We claim that the two anti-cheat systems we classify as rootkits raise severe concerns regarding their potential for malicious abuse and the associated privacy implications. However, although we do not classify the other two systems as rootkit-like, they nonetheless engage in practices that raise questions regarding their appropriateness and integrity.

While our work provides some first answers, it also highlights the need for further research on this topic to gain a more comprehensive understanding of different anti-cheat systems, their tactics, and how to best reconcile the need for robust anti-cheat solutions with respecting user privacy and system integrity. In particular, this article exclusively focussed on anti-cheat systems available on Microsoft Windows, disregarding other OSes. We thus see the need for future research into anti-cheat systems on other operating systems (e.g. EAC on Proton for Linux) comparing the applied methods to their Windows counterparts.

ACKNOWLEDGMENTS

We are grateful to Tobias Dam, Patrick Kochberger, Robert Luh, Martin Schmiedecker and the anonymous referees for suggesting numerous improvements to both the content and the presentation of this paper.

REFERENCES

- [1] Lynn Erla Beegle. 2007. Rootkits and Their Effects on Information Security. *Information Systems Security* 16, 3 (2007), 164–176. <https://doi.org/10.1080/10658980701402049>
- [2] Bill Blunden. 2013. *Rootkit Arsenal: Escape and Evasion in the Dark Corners of the System*. Jones & Bartlett Learning, Burlington, MA. <https://www.jblearning.com/catalog/productdetails/9781449626365>
- [3] "bright", "IDontCode", "irq0". 2021. EasyAntiCheat Exploit to Inject Unsigned Code into Protected Processes. Online. <https://blog.back.engineering/10/08/2021/>
- [4] "Broihon". 2018. Manual Mapping DLL Injection Tutorial - How To Manual Map. Online. <https://guidedhacking.com/threads/manual-mapping-dll-injection-tutorial-how-to-manual-map.10009/>
- [5] "Daax". 2020. Anticheat Faceit Bypass. Online. <https://guidedhacking.com/threads/anticheat-faceit-bypass.16113/post-89663?referrallcode=ON6pj>
- [6] "Daax", "iPower", "ajkhoury", "drew". 2020. How Anti-Cheats Detect System Emulation. Online. <https://secret.club/2020/04/13/how-anti-cheats-detect-system-emulation.html>
- [7] Sebastian Eresheim, Robert Luh, and Sebastian Schrittwieser. 2017. The Evolution of Process Hiding Techniques in Malware – Current Threats and Possible Countermeasures. *Journal of Information Processing* 25 (2017), 866–874. <https://doi.org/10.2197/ipsjip.25.866>
- [8] Hagen Fritsch. 2008. *Analysis and Detection of Virtualization-Vased Rootkits*. Bachelor's thesis. Technical University of Munich. <https://www.nm.ifi.lmu.de/pub/Fopras/frit08/PDF-Version/frit08.pdf>
- [9] "h4x0r2". 2023. Data Vanguard Is Grabbing to HWID Ban. Online. <https://www.unknowncheats.me/forum/valorant/567650-data-vanguard-grabbing-hwid-ban.html>
- [10] Intel. 2023. *Intel® 64 and IA-32 Architectures Software Developer's Manuals*. Intel. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [11] "iPower". 2020. CVEAC-2020: Bypassing EasyAntiCheat Integrity Checks. Online. https://secret.club/2020/04/08/eac_integrity_check_bypass.html
- [12] Xuxian Jiang. 2006. *Enabling Internet Worms and Malware Investigation and Defense Using Virtualization*. PhD thesis. Purdue University. <https://docs.lib.purdue.edu/dissertations/AAI3251634/>
- [13] Jestin Joy, Anita John, and James Joy. 2011. Rootkit Detection Mechanism: A Survey. In *Proceedings of the First International Conference on Parallel Distributed Computing Technologies and Applications (Tirunelveli) (PDCTA 2011/Communications in Computer and Information Science, vol. 203)*. Springer, Berlin/Heidelberg, 366–374. https://doi.org/10.1007/978-3-642-24037-9_36
- [14] Samuli Lehtonen. 2020. *Comparative Study of Anti-Cheat Methods in Video Games*. Master's thesis. University of Helsinki. <https://helda.helsinki.fi/items/b1141406-eb65-48a5-8922-d1b23d4cfe51>
- [15] Xiang Li, Yan Wen, Minhuan Huang, and Qiang Liu. 2011. An Overview of Bootkit Attacking Approaches. In *Proceedings of the 2011 Seventh International Conference on Mobile Ad-Hoc and Sensor Networks (Beijing) (MSN 2011)*. IEEE, New York, NY, 428–431. <https://doi.org/10.1109/MSN.2011.19>
- [16] Leian Liu, Zuanxing Yin, Yuli Shen, and Haitao Lin. 2012. Research and Design of Rootkit Detection Method. *Physics Procedia* 33 (2012), 852–857. <https://doi.org/10.1016/j.phpro.2012.05.145>
- [17] Anton Maario, Vinod Shukla, A. Ambikopathy, and Purushottam Sharma. 2011. Redefining the Risks of Kernel-Level Anti-Cheat in Online Gaming. In *Proceedings of the 2021 8th International Conference on Signal Processing and Integrated Networks (Noida) (SPIN 2021)*. IEEE, New York, NY, 676–680. <https://doi.org/10.1109/SPIN52536.2021.9566108>
- [18] Maxine Major. 2015. *A Taxonomic Evaluation of Rootkit Deployment, Behavior and Detection*. Master's thesis. University of Idaho. https://objects.lib.uidaho.edu/etd/pdf/Major_idaho_0089N_10700.pdf
- [19] Alex Matrosov, Eugene Rodionov, and Sergey Bratus. 2019. *Rootkits and Bootkits: Reversing Modern Malware and Next Generation Threats*. No Starch Press, San Francisco, CA. <https://nostarch.com/rootkits>
- [20] Egidius Mysliwicz. 2020. *Identifying Rootkit Stealth Strategies*. Bachelor's thesis. Radboud University. https://www.cs.ru.nl/bachelors-theses/2020/Egidius_Mysliwicz_1000796_Identifying_rootkit_stealth_strategies.pdf
- [21] Kyle Orland. 2020-04-14. Ring 0 of Fire: Does Riot Games' New Anti-Cheat Measure Go Too Far? *Ars Technica* (2020-04-14). <https://arstechnica.com/gaming/2020/04/ring-0-of-fire-does-riot-games-new-anti-cheat-measure-go-too-far/>
- [22] "Rake". 2015. Anticheat Battleye Bypass Overview. Online. <https://guidedhacking.com/threads/anticheat-battleye-bypass-overview.11602/>
- [23] "Rake". 2018. Anticheat Faceit Bypass. Online. <https://guidedhacking.com/threads/anticheat-faceit-bypass.16113/>
- [24] "Rake". 2020. How to Bypass EAC - Easy Anti Cheat. Online. <https://guidedhacking.com/threads/how-to-bypass-eac-easy-anti-cheat.15956/>
- [25] Caroline Andrea Rendenbach. 2022. *Anti-Cheating Measures in Video Games*. Bachelor's thesis. Technical University of Munich. <https://collab.dvb.bayern/download/attachments/77832800/main.pdf>
- [26] Riot Games. 2018. Riot's Approach to Anti-Cheat. Online. <https://technology.riotgames.com/news/riots-approach-anti-cheat>
- [27] Rolf Rolles. 2009. Unpacking Virtualization Obfuscators. In *Proceedings of the 3rd USENIX Workshop on Offensive Technologies (Montreal) (WOOT '09)*. USENIX Association, Berkeley, CA, 261–266. https://www.usenix.org/legacy/events/woot09/tech/full_papers/rolles.pdf
- [28] "SaltyPaster". 2021. How to Bypass EAC - Easy Anti Cheat. Online. <https://guidedhacking.com/threads/how-to-bypass-eac-easy-anti-cheat.15956/post-105040?referrallcode=ON6pj>
- [29] José Nuno Silva. 2022. *Towards Automated Server-side Video Game Cheat Detection*. Master's thesis. University of Porto. <https://repositorio-aberto.up.pt/bitstream/10216/142935/2/572983.pdf>
- [30] "Sinclairq". 2022. A Bank Vault's Self-Integrity Circumvented by an Underway Passage: How EasyAntiCheat's Driver Self-Integrity Can Be Compromised Through Call Hierarchy. Online. https://secret.club/2020/04/08/eac_integrity_check_bypass.html
- [31] UEFI Forum, Inc. 2019. *Unified Extensible Firmware Interface (UEFI) Specification*. Unified Extensible Firmware Interface (UEFI) Forum. <https://uefi.org/specifications>
- [32] Nikos Virvilis and Dimitris Gritzalis. 2013. The Big Four – What We Did Wrong in Advanced Persistent Threat Detection?. In *Proceedings of the 2013 International Conference on Availability, Reliability and Security (Regensburg) (ARES 2013)*. IEEE, New York, NY, 248–254. <https://doi.org/10.1109/ARES.2013.32>
- [33] "vmcall". 2019. BattlEye Anti-Cheat: Analysis and Mitigation. Online. <https://secret.club/2019/02/10/battleye-anticheat.html>
- [34] "vmcall". 2020. BattlEye Hypervisor Detection. Online. <https://secret.club/2020/01/12/battleye-hypervisor-detection.html>
- [35] "whatacoolwitch". 2021. Uninstalling and Disabling Riot Vanguard. Online. <https://support-valorant.riotgames.com/hc/en-us/articles/360044648213-Uninstalling-and-Disabling-Riot-Vanguard>
- [36] "whatacoolwitch". 2022. What Is Vanguard? Online. <https://support-valorant.riotgames.com/hc/en-us/articles/360046160933-What-is-Vanguard>
- [37] "Xyrem". 2023. In-Depth Analysis on Valorant's Guarded Regions. Online. <https://reversing.info/posts/guardedregions/>
- [38] "yousif". 2020. Bypassing BattlEye from User-Mode. Online. https://secret.club/2020/02/26/be_umode.html